

Flooding Consensus Algorithm in Code

Danilo Kaćanski

Faculty of Technical Sciences, Novi Sad
Applied algorithms in Control Systems

May 2026.

Process Contract

Messages between Nodes

week03_04.../process/message.go

```
type Message struct {  
    From ProcessID  
    To ProcessID  
    Type string  
    Data any  
    Meta map[string]any  
}
```

- Data any: generic payload for any algorithm
- Meta: internal link/crypto metadata
- Transport layer moves messages without understanding their meaning

week03_04.../process/process.go

```
type Process interface {  
    ID() ProcessID  
  
    Run(  
        ctx context.Context,  
        inbox <-chan Message,  
        send func(Message),  
    )  
}
```

- Every node runs as its own goroutine
- inbox: incoming messages
- send: emit outgoing messages
- ctx: runtime-controlled shutdown

What happens after `send(msg)`?

- ① runtime logs the message
- ② link stack applies failures/retransmissions
- ③ surviving messages enter the network channel
- ④ router eventually delivers them to inboxes

Important

Processes never communicate directly. Everything flows through the runtime and link stack.

Types in Flooding Consensus

Proposal Values and Message Kinds

Core consensus types

```
type ProposalValue string
const (
    KindPropose = "INIT_PROPOSE"
    KindProposal = "PROPOSAL"
    KindDecided = "DECIDED"
    KindCrashDetected = "CRASH_DETECTED"

    ConsensusType = "CONSENSUS"
)
```

- ProposalValue: values processes propose and decide
- Each Kind maps directly to one event in Algorithm 5.1
- ConsensusType: outer network-level protocol label

ConsensusMessage Payload

ConsensusMessage

```
type ConsensusMessage struct {  
    Kind string  
    Round int  
    From process.ProcessID  
    Values []ProposalValue  
    Value ProposalValue  
    Crashed process.ProcessID  
}
```

Important

ConsensusMessage is a single payload format for multiple consensus events. The Kind field decides which fields are meaningful.

Mapping to Algorithm 5.1

Kind	Used Fields	Book Notation
INIT_PROPOSE	Value	$\langle \textit{Propose} \mid v \rangle$
PROPOSAL	Round, From, Values	$[\textit{PROPOSAL}, r, ps]$
DECIDED	From, Value	$[\textit{DECIDED}, v]$
CRASH_DETECTED	Crashed	$\langle \textit{Crash} \mid p \rangle$

Key Idea

Every pseudocode event from the book becomes a typed Go message.

Important Helper Functions

Core helpers

`MinValue(vals)`

`ProcessSetEqual(a, b)`

`SortedValues(vals)`

`CopyValues(vals)`

- `MinValue`: implements

$$decision := \min(proposals[round])$$

- `ProcessSetEqual`: checks

$$receivedfrom[r] = receivedfrom[r - 1]$$

- `SortedValues`: deterministic ordering for traces
- `CopyValues`: prevents shared mutable state

Consensus Node

ConsensusNode Structure

consensus/node.go

```
type ConsensusNode struct {  
    id process.ProcessID  
    allIDs []process.ProcessID  
    recorder *DecisionRecorder  
}
```

- id: identity of this node
- allIDs: all participants in the consensus instance
- recorder: shared result store for final checks

Important

The algorithm state is not stored in the struct. It lives inside `Run(...)` as local goroutine state.

Init Block of Run()

Algorithm state inside Run()

```
correct := make(map[process.ProcessID]bool)
for _, pid := range n.allIDs {
    correct[pid] = true
}
round := 1
var decision ProposalValue
receivedfrom := make(map[int]map[process.ProcessID]bool)
receivedfrom[0] = make(map[process.ProcessID]bool)
for _, pid := range n.allIDs {
    receivedfrom[0][pid] = true
}
proposals := make(map[int]map[ProposalValue]bool)
proposals[1] = make(map[ProposalValue]bool)
```

Init Block: Pseudocode to Go

Book notation	Go implementation
$correct := \perp$	map with all process IDs set to true
$round := 1$	<code>round := 1</code>
$decision := \perp$	empty string means undecided
$receivedfrom[0] := \perp$	round 0 contains all process IDs
$proposals[1] := \emptyset$	empty proposal set for round 1

Why $receivedfrom[0] = \perp$?

In round 1, the algorithm compares:

$$receivedfrom[1] = receivedfrom[0]$$

So round 1 can decide only if the node heard from the full initial process set.

Local State and ensureRound

Lazy round initialization

```
ensureRound := func(r int) {  
    if receivedfrom[r] == nil {  
        receivedfrom[r] = make(map[process.ProcessID]bool)  
    }  
    if proposals[r] == nil {  
        proposals[r] = make(map[ProposalValue]bool)  
    }  
}
```

- Messages for future rounds may arrive early.
- Missing map entries would otherwise be `nil`.
- `ensureRound(r)` creates storage before using round r .
- Because this state is local to one Run goroutine, no mutex is needed.

Core termination logic

```
if decision != "" {return}
ensureRound(round)
for pid := range correct {
    if !receivedfrom[round][pid] {return}
}
if ProcessSetEqual(receivedfrom[round], receivedfrom[round-1]) {
    decision = MinValue(proposals[round])
    recorder.RecordDecision(n.id, decision)
    Broadcast(send, n.id, allIDs, DECIDED)
} else {
    round++
    proposals[round] = CopyValues(proposals[round-1])
    Broadcast(send, n.id, allIDs, PROPOSAL)
}
```


What does `maybeAdvanceOrDecide()` Check?

1. Has this node already decided?

If yes, do nothing. Consensus integrity forbids deciding twice.

2. Have we heard from everyone believed correct?

$$\text{correct} \subseteq \text{receivedfrom}[\text{round}]$$

If not, the node must keep waiting.

3. Is the sender set stable across two rounds?

$$\text{receivedfrom}[\text{round}] = \text{receivedfrom}[\text{round} - 1]$$

If yes, decide. If not, move to the next round and flood again.

Step 5: Main Message Loop

Node goroutine loop

```
for {
    select {
    case <-ctx.Done():
        return
    case msg, ok := <-inbox:
        if !ok {return}
        cm, ok := msg.Data.(ConsensusMessage)
        if !ok {continue}
        switch cm.Kind {
        case KindPropose:
        case KindProposal:
        case KindCrashDetected:
        case KindDecided:
        }
    }
}
```

Message Handlers I

KindPropose

- starts the protocol for this node
- adds the node's own value to `proposals[1]`
- broadcasts:

`[PROPOSAL, 1, proposals[1]]`

KindProposal

- records sender in:

`receivedfrom[r][sender]`

- merges received values into:

`proposals[r][value]`

- if $r = \text{round}$, calls `maybeAdvanceOrDecide()`

KindCrashDetected

- removes crashed process from correct
- may unblock:

$$\text{correct} \subseteq \text{receivedfrom}[\text{round}]$$

- then calls `maybeAdvanceOrDecide()`

KindDecided

- if sender is still considered correct and this node is undecided:
 - adopt the decision
 - record it
 - rebroadcast DECIDED

Best-Effort Broadcast

Best-Effort Broadcast

consensus/broadcast.go

```
func Broadcast(  
    send func(process.Message), // send callback, provided by runtime  
    from process.ProcessID, // sender process ID  
    allIDs []process.ProcessID, // all process IDs  
    payload ConsensusMessage, // message to broadcast  
) {  
    for _, to := range allIDs {  
        msg := process.NewMessage(from, to, ConsensusType, payload)  
        send(msg)  
    }  
}
```

Where do the Broadcast Guarantees come from?

- **Broadcast function**
 - only provides the *send to all* loop
- **PerfectLink underneath**
 - removes duplicates
 - retransmission helps delivery
 - prevents created/invalid deliveries
- **Runtime**
 - routes each unicast message to the correct inbox

Perfect Failure Detector

Perfect Failure Detector Oracle

- Flooding Consensus assumes a perfect failure detector:

$$\langle \mathcal{P}, \text{Crash} \mid p \rangle$$

- In code, this is simulated with planned crash events.
- The oracle knows which process crashes and when to notify others.

Key Idea

The consensus algorithm only sees `CRASH_DETECTED`. It does not care how detection was implemented.

Crash Event

failure_detector.go

```
type CrashEvent struct {  
    Crashed process.ProcessID  
    NotifyAt time.Duration  
}
```

- Crashed: process that stops
- NotifyAt: when other nodes are notified

Crash Injection Logic

Simplified

```
time.Sleep(ev.NotifyAt)

crashFM.CrashProcess(ev.Crashed)
rt.CrashProcess(ev.Crashed)

for _, to := range allIDs {
    if to != ev.Crashed {
        rt.Inject(CRASH_DETECTED(ev.Crashed))
    }
}
```

Important

`rt.Inject` is used because crash detection is a control event, not a normal network message.

Real vs Simulated PFD

Real PFD	Simulator Oracle
heartbeats	planned crashes
timeouts	fixed notification time
detects from network behavior	knows crash schedule

Teaching Goal

We focus on how consensus reacts to crash detection, not on implementing heartbeat-based detection.

Consensus Checkers

checkers.go

```
type DecisionRecorder struct {  
    mu sync.Mutex  
    decisions map[process.ProcessID]ProposalValue  
    proposed map[process.ProcessID]ProposalValue  
    crashed map[process.ProcessID]bool  
}
```

- Stores structured simulation results.
- Shared by all consensus nodes.
- Protected by a mutex because nodes run as goroutines.
- Used after the simulation to check consensus properties.

Four Consensus Property Checks

Checker	What it verifies
CheckAgreement	correct processes did not decide differently
CheckValidity	decided value was actually proposed
CheckIntegrity	each process has only one decision value
CheckTermination	every correct process eventually decided

Important

Crashed processes are ignored for agreement and termination, because the consensus properties are required for correct processes.

After the simulation

```
snapshot := recorder.Snapshot()
```

```
RunAllChecks(snapshot, allIDs)
```

- `Snapshot()` creates a safe copy of recorder state.
- Each checker returns:

PASS or *FAIL*

- Failed checks include a short explanation.

Runnable Examples

Example 1: No Failure

- Processes:

$$\{A, B, C\}$$

- Proposed values:

$$A, B, C$$

- No process crashes.
- Every node receives proposals from all nodes in round 1:

$$receivedfrom[1] = receivedfrom[0]$$

- All nodes decide:

$$\min\{A, B, C\} = A$$

What This Shows

Without failures, Flooding Consensus decides in the first round.

Example 2 vs Example 3

	Example 2	Example 3
Scenario	one crash	partial message before crash
Crashed node	node-D	node-C
Crash behavior	sends nothing	sends only to node-A
Round 1	everyone blocked	node-A can decide
Decision path	round 2 needed	DECIDED relay used

Main Difference

Example 2 demonstrates the **advance-to-next-round** branch. Example 3 demonstrates the **adopt-DECIDED** branch.

Example 2: One Crash, Round 2 Required

- Processes:

$$\{A, B, C, D\}$$

- Node-D is registered, but never sends a proposal.
- Nodes A, B, C receive only from each other:

$$receivedfrom[1] = \{A, B, C\}$$

- But initially:

$$receivedfrom[0] = \{A, B, C, D\}$$

- Since the sets differ, they cannot decide in round 1.
- They advance to round 2 and decide after the sets stabilize.

Example 3: DECIDED Relay

- Processes:

$$\{A, B, C\}$$

- Node-C sends its proposal only to node-A, then crashes.
- Node-A has:

$$receivedfrom[1] = \{A, B, C\} = receivedfrom[0]$$

- Node-A decides in round 1 and broadcasts DECIDED(A).
- Node-B did not receive C's proposal, but it receives DECIDED(A).
- Node-B adopts the decision directly.

What This Shows

A process does not always need to finish the same round. It can safely adopt a decision already broadcast by another correct process.